CENTRO UNIVERSITÁRIO JORGE AMADO
BACHAREL EM CIÊNCIA DA COMPUTAÇÃO

JEAN ALEX SANTOS DE LIMA
RENATA DE SOUZA SILVA

AUTOCOMPLETAR POR PREFIXO: DIAGNÓSTICO E PRIMEIRA OTIMIZAÇÃO

Salvador/BA
2026

1. Descrição do problema

Escolhemos o tema 3, autocompletar por prefixo. É o recurso que, enquanto a pessoa digita, mostra as palavras do dicionário que começam com o que ela já escreveu, como acontece no teclado do celular ou na barra de busca de um site. O código-base fornecido resolve isso da forma mais direta: a cada tecla, percorre a lista inteira de palavras e separa as que começam com o prefixo atual.

Com poucas palavras a abordagem funciona bem, mas o custo cresce na mesma proporção do dicionário e é pago de novo a cada tecla, justamente numa operação que precisa responder rápido para não atrapalhar a digitação. Nesta entrega medimos a versão original, explicamos de onde vem o custo e aplicamos uma primeira otimização: ordenar o dicionário uma única vez e encontrar as sugestões por busca binária. A trie, estrutura indicada para o tema, fica para a Entrega 2, e os números obtidos aqui ajudam a justificar essa escolha.

2. Versão original e análise de complexidade

A versão de partida é a do enunciado, mantida sem alterações em `original.py`:

```
def sugerir(palavras, prefixo):
    resultado = []
    for p in palavras:
        if p.startswith(prefixo):
            resultado.append(p)
    return resultado
```

Consideramos n o número de palavras do dicionário, m o tamanho do prefixo digitado e r a quantidade de sugestões devolvidas. O laço executa exatamente n iterações e não tem nenhuma condição de parada antecipada: mesmo depois de encontrar todas as palavras possíveis, segue até o fim da lista. Em cada iteração, `startswith` compara no máximo m caracteres e o `append` custa O(1) amortizado. Assim, uma consulta custa O(n·m) no pior caso. Como r nunca passa de n, o trabalho de montar a resposta não altera essa ordem. O espaço extra é O(r), ocupado apenas pela lista de resultado.

Na prática, a comparação costuma parar logo no primeiro ou segundo caractere, porque a maioria das palavras já difere do prefixo no início. Por isso o comportamento observado é praticamente linear, Θ(n) por tecla, e as medições confirmam: cada vez que o dicionário fica dez vezes maior, o tempo por tecla também fica cerca de dez vezes maior (Tabela 1).

O que mais pesa é a repetição. Digitar uma palavra de L letras gera L consultas, e cada uma refaz a varredura completa, o que dá O(L·n) só para uma palavra. Para escrever “casaco” num dicionário de um milhão de palavras, o programa examina seis milhões de palavras, sendo que, a partir da segunda letra, as sugestões são sempre um subconjunto das que já tinham sido encontradas na tecla anterior.

3. Gargalos identificados

O primeiro gargalo é a varredura completa a cada tecla. A função não guarda nada sobre o dicionário entre uma consulta e outra, então cada tecla paga O(n) novamente, mesmo que o dicionário não tenha mudado.

O segundo é que a organização das palavras não é aproveitada. Como a lista não tem ordem definida, palavras com o mesmo prefixo podem estar em qualquer posição, e a única forma de garantir que nenhuma ficou de fora é olhar todas. É esse o ponto que a otimização desta entrega ataca.

O terceiro é o custo do próprio interpretador. O laço roda em Python puro, com uma chamada de método e um possível *append* por palavra. Reescrever o laço como list comprehension reduziria a constante, mas o crescimento continuaria linear, por isso não tratamos essa troca como a otimização principal.

4. Primeira otimização: dicionário ordenado e busca binária

Se o dicionário estiver em ordem alfabética, todas as palavras que começam com um prefixo p ficam juntas, formando um bloco contínuo na lista. Esse bloco começa na primeira palavra maior ou igual a p e termina logo antes da primeira palavra maior ou igual a p seguido de `chr(0x10FFFF)`, o maior caractere Unicode. Qualquer palavra que comece com p é menor que esse limite, e qualquer palavra que não comece com p fica fora do intervalo. As duas posições são encontradas com `bisect_left`, da biblioteca padrão, e a resposta é a fatia da lista entre elas:

```
from bisect import bisect_left

MAIOR_CARACTERE = chr(0x10FFFF)


class Autocompletar:
    def __init__(self, palavras):
        self.palavras = sorted(palavras)

    def sugerir(self, prefixo):
        inicio = bisect_left(self.palavras, prefixo)
        fim = bisect_left(self.palavras, prefixo + MAIOR_CARACTERE, inicio)
        return self.palavras[inicio:fim]
```

A ordenação acontece uma única vez, quando o índice é criado. Ela faz O(n log n) comparações de palavras, cada uma com custo proporcional ao tamanho médio L das palavras, totalizando O(L·n log n) de tempo e O(n) de espaço para a cópia ordenada. Cada consulta faz duas buscas binárias, com O(log n) comparações de até m caracteres, e depois copia as r sugestões da fatia. O custo por tecla passa a ser O(m·log n + r), contra O(n·m) da versão original. A dependência do tamanho do dicionário deixa de ser linear e passa a ser logarítmica. O termo r continua presente, porque qualquer solução que devolva todas as sugestões precisa ao menos copiá-las.

Quanto à correção, a função original devolve as sugestões na ordem em que aparecem no dicionário, e a nova devolve em ordem alfabética. O conjunto de palavras é o mesmo, inclusive com repetições, e a ordem alfabética é a esperada num autocompletar. Se o dicionário original já estiver ordenado, as duas saídas são idênticas. O arquivo `test_autocompletar.py` compara a saída da versão otimizada com a da original ordenada em todas as teclas de 50 palavras digitadas num dicionário de 20 mil palavras, e também em casos de borda: prefixo vazio, prefixo inexistente, prefixo maior que todas as palavras, palavras repetidas, acentos, maiúsculas e dicionário vazio. Todos os testes passam.

5. Metodologia de medição

Para ter dicionários de vários tamanhos com as mesmas características, geramos as palavras de forma sintética (`dados.py`), com semente fixa para que o experimento possa ser repetido. Cada palavra junta de duas a cinco sílabas no formato consoante e vogal, o que produz palavras como “cucero”, “nobi” e “gidavigece”. Diferente de letras sorteadas ao acaso, esse formato faz muitas palavras compartilharem o início, como acontece num dicionário de verdade.

Para representar o uso real, sorteamos 20 palavras do dicionário e geramos as consultas que alguém faria ao digitá-las letra por letra, o que deu entre 120 e 144 consultas por tamanho. As duas versões respondem exatamente às mesmas consultas. O tempo foi medido com a função `mede`, sugerida no enunciado, que usa `time.perf_counter` e guarda o melhor de três execuções. Dividimos o tempo total pelo número de consultas para obter o tempo por tecla. A construção do índice (a ordenação) foi medida à parte, já que só acontece uma vez.

Todos os testes rodaram na mesma máquina, um notebook com processador AMD Ryzen de 12 threads, Windows 11 e Python 3.14.2. O script `benchmark.py` repete o experimento completo e grava os números em `resultados.csv` e o gráfico em `grafico.png`.

6. Resultados

Tabela 1. Tempo médio por tecla e custo de construção do índice

Palavras (n)	Sugestões por tecla	Original (ms/tecla)	Otimizado (ms/tecla)	Ganho	Ordenação (ms)	Teclas para compensar
1.000	15	0,0287	0,000529	54×	0,134	5
10.000	158	0,250	0,000798	314×	1,45	6
100.000	1.538	2,58	0,00441	584×	23,8	10
1.000.000	13.021	26,0	0,193	135×	401	16

A última coluna indica quantas teclas são necessárias para que a economia por consulta compense o tempo gasto ordenando o dicionário.

A versão original cresce de forma linear, como previsto: passa de 0,0287 ms por tecla com mil palavras para 26,0 ms com um milhão. A versão otimizada fica abaixo de 0,2 ms por tecla em todos os tamanhos testados, com ganhos entre 54 e 584 vezes. No gráfico, em escala logarítmica nos dois eixos, a reta da versão original tem inclinação próxima de 1, característica de O(n), enquanto a curva da versão otimizada fica entre duas e três ordens de grandeza abaixo.

O custo de ordenar o dicionário não anula o ganho. Com um milhão de palavras, a ordenação leva cerca de 0,4 s, mas cada tecla economiza perto de 26 ms, então o índice se paga em umas 16 teclas, menos do que digitar três palavras. Num autocompletar o dicionário é carregado uma vez e consultado milhares de vezes, de modo que esse custo inicial fica diluído.

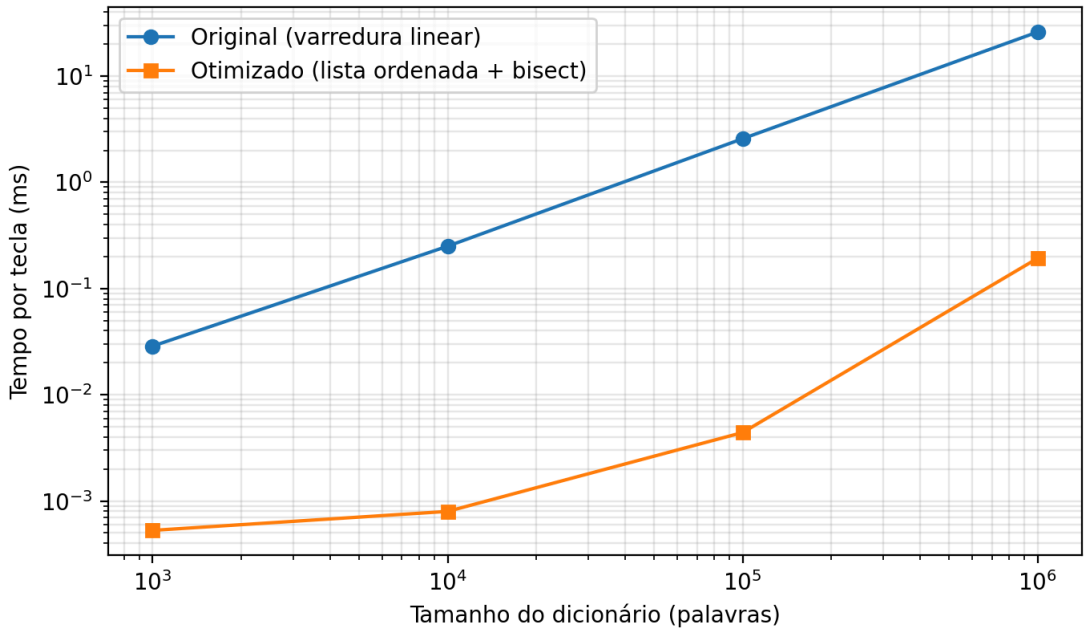


Figura 1. Tempo por tecla em função do tamanho do dicionário (escala log-log)

7. Discussão: o gargalo mudou de lugar

Um resultado chamou atenção: o ganho sobe até 584 vezes com cem mil palavras e cai para 135 vezes com um milhão. A versão otimizada, que deveria crescer devagar, ficou 44 vezes mais lenta quando o dicionário cresceu 10 vezes. Para entender o motivo, medimos separadamente só as duas buscas binárias, sem copiar a fatia.

Tabela 2. Parcela do tempo da versão otimizada gasta em cada etapa

Palavras (n)	Só busca binária (ms/tecla)	Consulta completa (ms/tecla)	Tempo gasto copiando sugestões
1.000	0,00035	0,00053	34,3%
10.000	0,00037	0,00080	53,2%
100.000	0,00058	0,0044	86,9%
1.000.000	0,00085	0,19	99,6%

A busca binária cresce de forma logarítmica, como esperado: vai de 0,00035 ms para 0,00085 ms por tecla enquanto o dicionário cresce mil vezes. Quase todo o tempo restante está em copiar as sugestões. A primeira letra digitada casa com cerca de 1/13 do dicionário, já que as palavras começam com uma de 13 consoantes, e a segunda, com cerca de 1/65. Com um milhão de palavras, isso dá dezenas de milhares de sugestões nas primeiras teclas e uma média de 13 mil por tecla. A Tabela 1 mostra que essa média cresce na mesma proporção que n. Ou seja, o termo log n da análise já não é o problema: o custo agora é dominado por r, que cresce junto com o dicionário nas primeiras teclas.

Há também um custo de memória. A lista ordenada é uma cópia das referências do dicionário, o que acrescenta O(n) de espaço, embora as strings em si não sejam duplicadas. Incluir uma palavra nova exige inserção ordenada, que custa O(n) por causa do deslocamento dos elementos. Para um dicionário que muda pouco isso não chega a ser um problema, mas é uma limitação a registrar.

8. Próximos passos (Entrega 2)

A Tabela 2 deixa claro que melhorar só a localização do prefixo já não traz muito ganho, porque a busca binária custa menos de um milésimo de milissegundo. Para a Entrega 2, pretendemos implementar a trie prevista no tema, que encontra o nó do prefixo em O(m) sem depender de n, e combiná-la com um limite de k sugestões por consulta, como fazem os autocompletares reais, que mostram de cinco a dez opções. Com isso a consulta passa a custar O(m + k), atacando o termo r que hoje domina o tempo.

Também vamos medir o consumo de memória com `tracemalloc`, já que a trie costuma ocupar bem mais espaço que a lista ordenada, e esse deve ser o principal trade-off de tempo e espaço a discutir. Como limitar as sugestões muda a saída, os testes passarão a comparar as k primeiras sugestões em ordem alfabética com as da versão original.

9. Conclusão

O código original tem custo O(n·m) por tecla e repete a varredura completa do dicionário a cada letra digitada, chegando a 26 ms por tecla com um milhão de palavras. Ordenar o dicionário uma vez e localizar o bloco de sugestões com busca binária reduziu esse custo para O(m·log n + r), com ganhos medidos entre 54 e 584 vezes e as mesmas sugestões da versão original. As medições também mostraram que, nos dicionários maiores, o gargalo passou a ser a quantidade de sugestões devolvidas, e é nesse ponto que a Entrega 2 vai se concentrar.